CS F364: Design & Analysis of Algorithms

Quiz 2 – June 4, 2026

Coverage: Lectures 1–10

> **Note**
>
> **Instructions:**
>
> - Time: 15 minutes.
> - Write your answers clearly on paper, scan, and upload to Google Classroom.
> - Show all steps for full credit.

Question

Consider a modification of standard Merge Sort, called **3-way Merge Sort**, which works as follows: - **Divide:** Divide the input array $A[1..n]$ into three subarrays of size roughly $n/3$. - **Conquer:** Recursively sort each of the three subarrays. - **Combine:** Merge the three sorted subarrays into a single sorted array.

(a) [1 Mark] How many comparisons are required in the worst case to merge three sorted lists of combined size n into a single sorted list? Explain briefly. (Use exact values, including constants)

(b) [1 Mark] Write the recurrence relation for the worst-case running time $T(n)$ of 3-way Merge Sort and state its overall asymptotic complexity in Θ notation.

(c) [3 Marks] Compare the running time of 3-way Merge Sort to standard 2-way Merge Sort: 1. What is the exact running time in worst case of 2-way merge sort? 2. What is the exact running time of 3-way merge sort? 3. Which version is more comparison-efficient? (Hint: $\log_3 n = \frac{\log_2 n}{\log_2 3} \approx 0.63 \log_2 n$)

Solution

Part (a) — Merging three sorted lists [1 Mark]

To merge three sorted lists of combined size n, we compare the front elements of the (up to) three non-empty lists at each step.

- When all 3 lists are non-empty: 2 comparisons to find the minimum
- When only 2 lists remain: 1 comparison
- The last element requires no comparison

In the worst case, all three lists are exhausted at roughly the same time. Each of the n elements (except the last one) needs $3 - 1 = 2$ comparisons to determine its rank.

$$\boxed{2\,(n-1) \text{ comparisons in the worst case}}$$

Part (b) — Recurrence [1 Mark]

Divide: Split array into 3 parts of size $n/3 \to \Theta(1)$.

Conquer: Recursively sort each part $\to 3\,T(n/3)$.

Combine: Merge 3 sorted lists of total size $n \to 2(n-1) = \Theta(n)$.

$$T(n) = 3\,T(n/3) + \Theta(n), \qquad T(1) = \Theta(1)$$

Master Theorem: $a = 3$, $b = 3$, $f(n) = \Theta(n)$.
$\log_b a = \log_3 3 = 1$, so $f(n) = \Theta(n^{\log_b a}) = \Theta(n^1) \to$ **Case 2**.

$$\boxed{T(n) = \Theta(n \log n)}$$

Part (c) — Comparison with 2-way Merge Sort [3 Marks]

1. Exact running time of 2-way Merge Sort

Merge of 2 sorted lists of size n needs $n-1$ comparisons.

$$T_2(n) = 2\,T_2(n/2) + (n-1), \qquad T_2(1) = 0$$

Solving:

$$\boxed{T_2(n) = n\log_2 n - n + 1 \;\approx\; n\log_2 n}$$

2. Exact running time of 3-way Merge Sort

Merge of 3 sorted lists of size n needs $2(n-1)$ comparisons.

$$T_3(n) = 3\,T_3(n/3) + 2(n-1), \qquad T_3(1) = 0$$

Solving via recursion tree:

$$\begin{aligned} T_3(n) &= \sum_{k=0}^{\log_3 n - 1} (2n - 2\cdot 3^k) \\ &= 2n\log_3 n - (n-1) \\ &= 2n \cdot \frac{\log_2 n}{\log_2 3} - n + 1 \\ &\approx 1.26\, n\log_2 n - n + 1 \end{aligned}$$

$$\boxed{T_3(n) = 2n\log_3 n - n + 1}$$

3. Which is more comparison-efficient?

Compare the leading terms:

Version	Comparisons (approx)
2-way	$n \log_2 n$
3-way	$1.26\, n \log_2 n$

2-way Merge Sort is more comparison-efficient

3-way Merge Sort performs about **26% more comparisons** because while the recursion depth decreases ($\log_3 n$ vs $\log_2 n$), the merge step costs $2(n-1)$ instead of $(n-1)$, which more than offsets the shallower recursion.